

◆ Project Description:

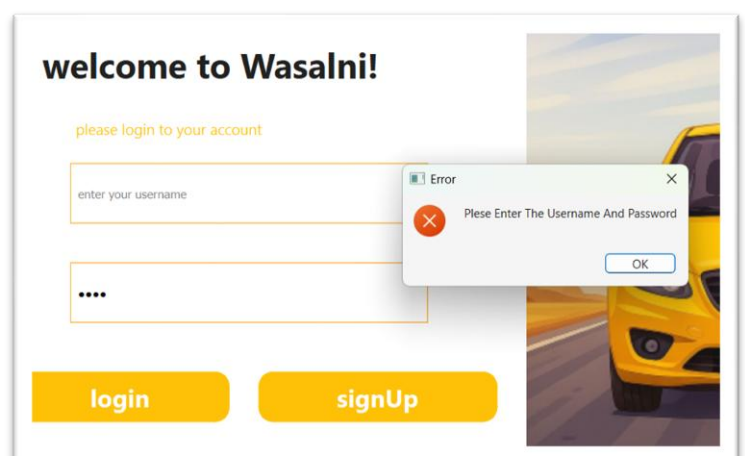
This project is a C++ system designed to simulate a smart navigation system similar to "Wasalni". It models cities and the roads connecting them using a graph data structure with adjacency lists. The system allows users to manage graphs, including adding and removing cities (nodes) and roads (edges), and to perform various graph operations such as Breadth-First Search (BFS), Depth-First Search (DFS), and Dijkstra's algorithm for both the shortest and fastest routes. It also includes a persistent file storage mechanism and a user detailed history of searched paths. The project features a graphical user interface built with Qt, enabling easy interaction with the system through login/sign-up screens, graph visualization, and route search tools.

◆ Input/Output scenarios:

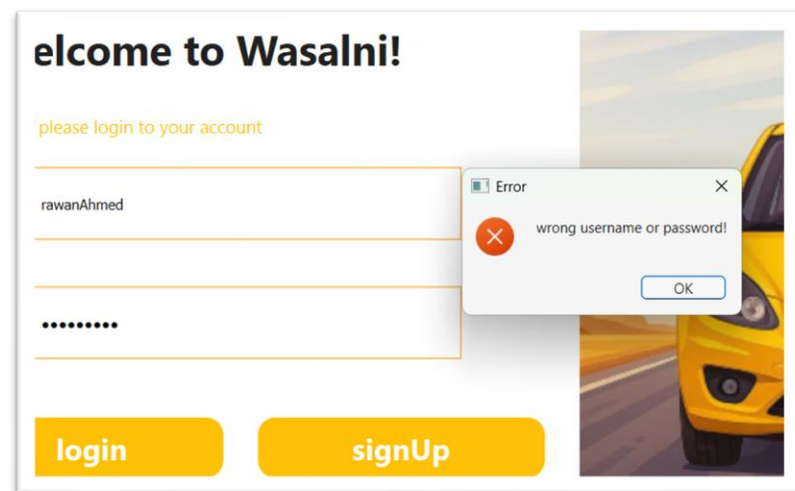
Login/signup Page:

The Login Page provides fields for users to enter a username and password, with two main actions: Login and Sign Up. Upon clicking either button, the following validations and behaviors are applied:

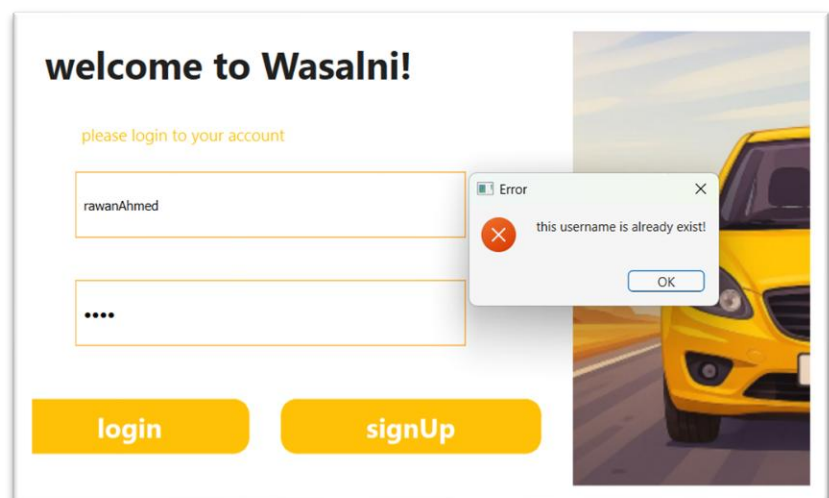
- Both username and password fields must be filled; otherwise, an error message is displayed.



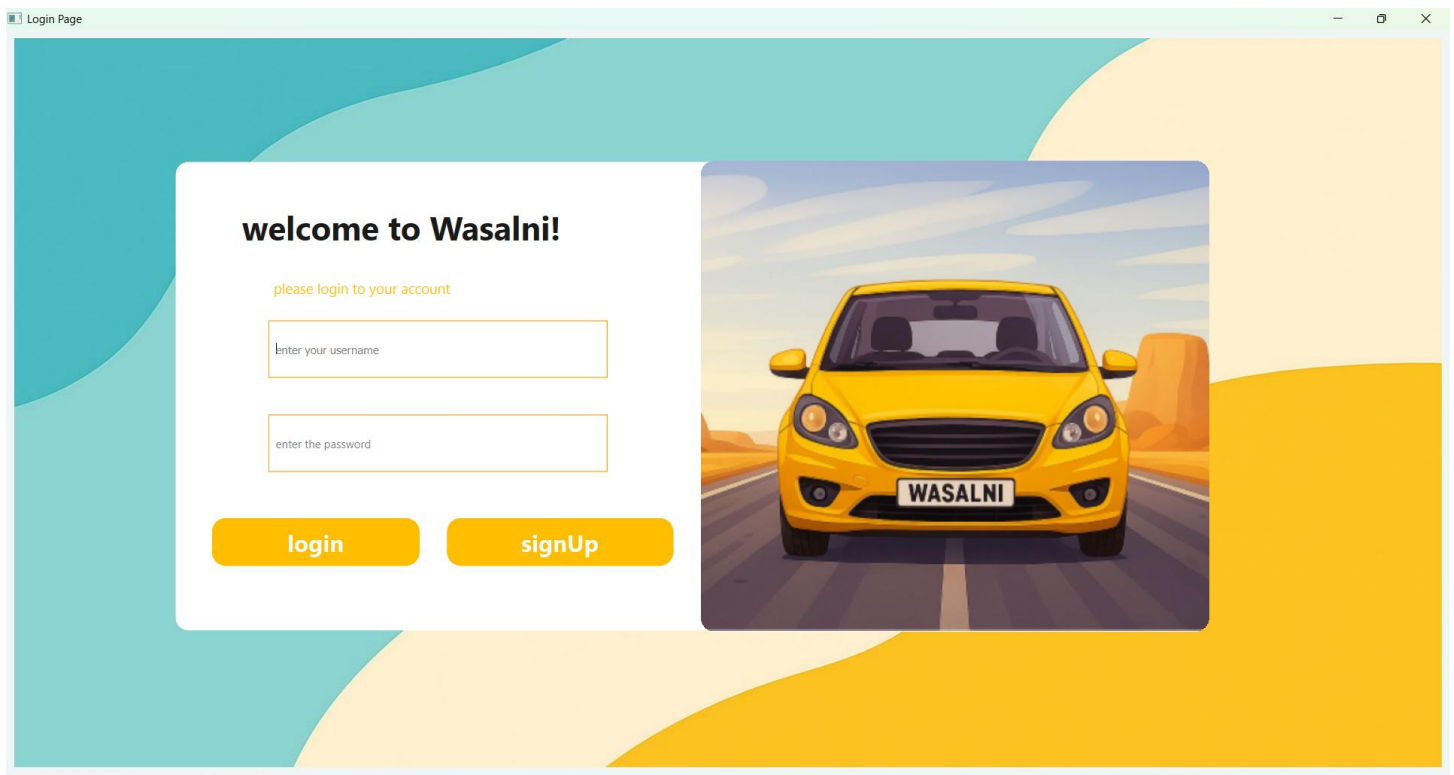
- On Login:
 - If the credentials match an existing account, the user is redirected to the Home Page.
 - If credentials are incorrect, an error message appears indicating invalid login.



- On Sign Up:
 - If the username is new, a user account is created, and the user is redirected to the Home Page.
 - If the username already exists, an error message notifies the user.

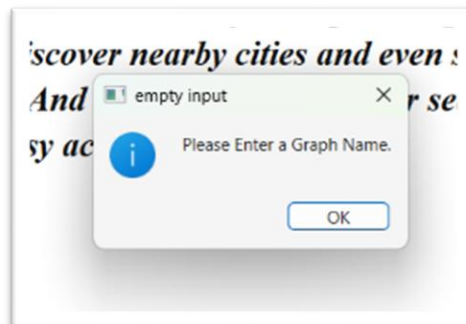
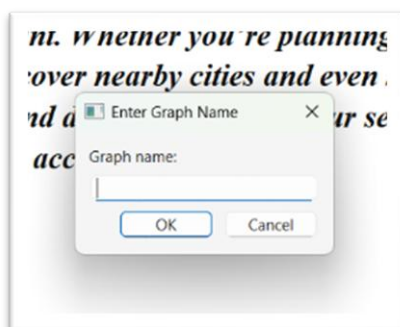


All interactions are confirmed through pop-up messages, and the window launches in full-screen mode for a better visual experience.

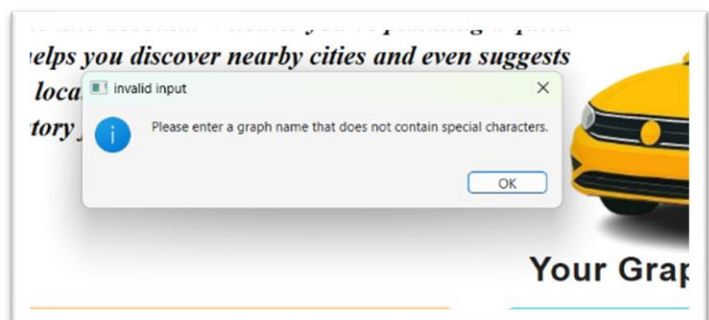
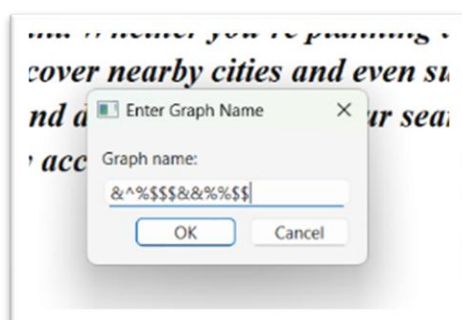


◆ Home Page:

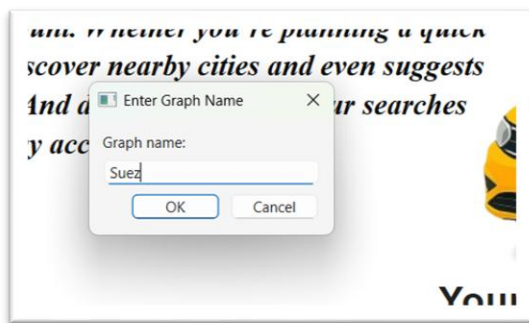
The Home Page allows users to view available graphs (system and personal), add new graphs, access their history, and log out. When adding a new graph, the system performs several validations: the graph name must not be empty:



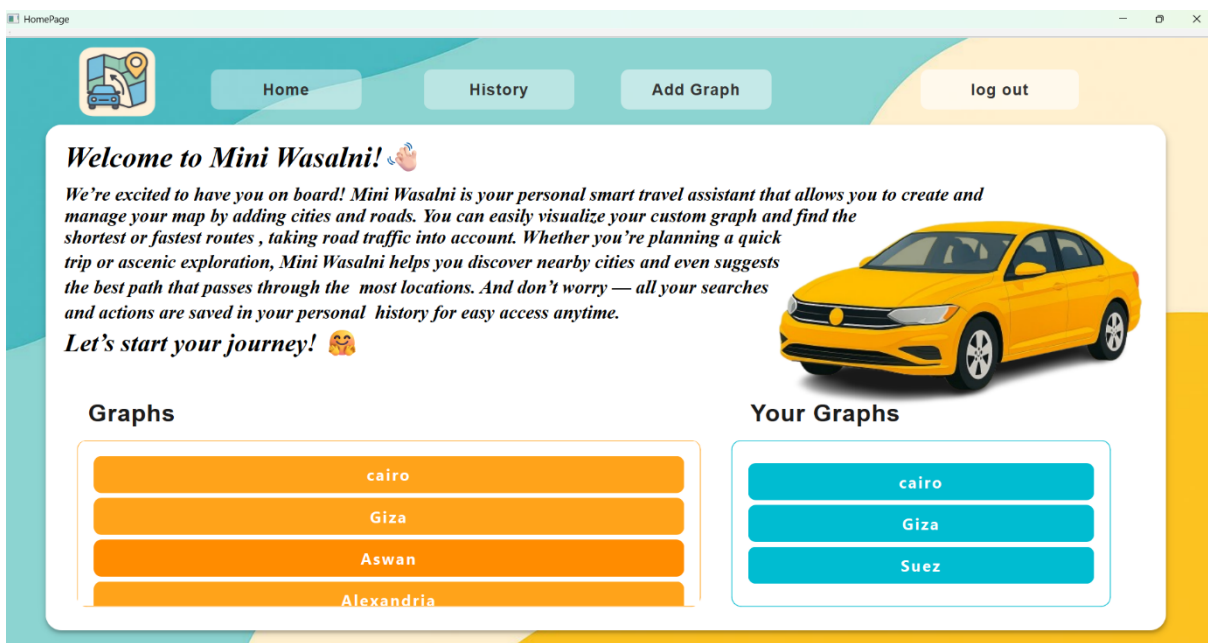
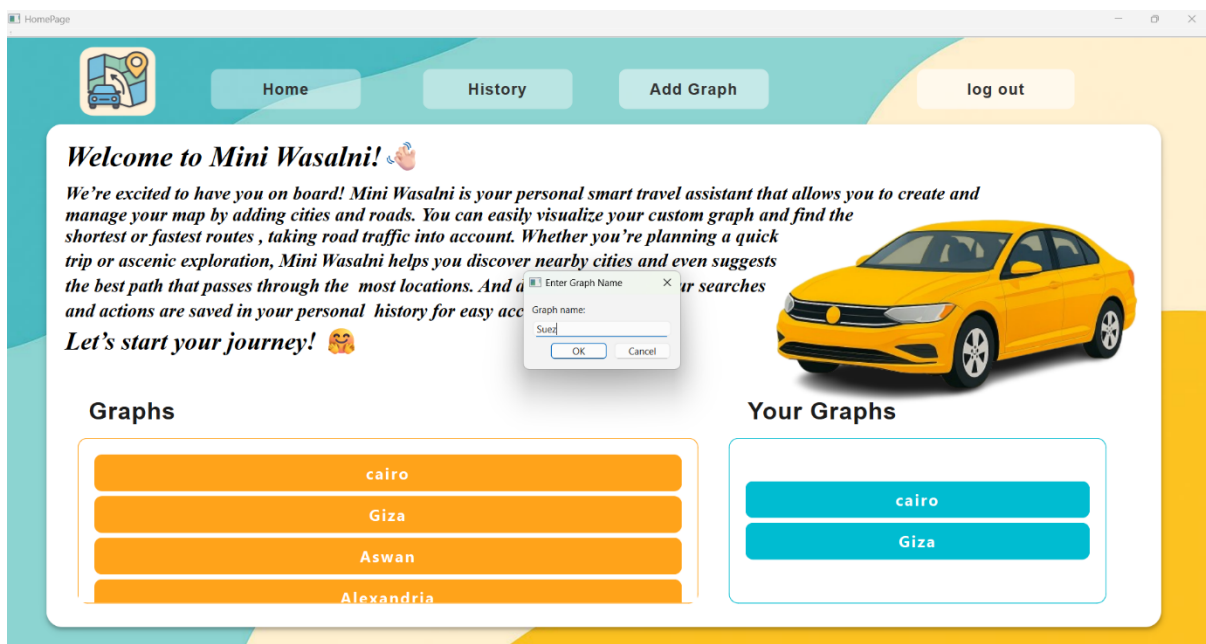
should not contain any special characters:



and must be unique (i.e., not already existing in the system):



Upon selecting a graph, the user is redirected to its corresponding page for further interaction.



◆ Graph Page:

The Graph Page allows user to do some modifications on the graph (if owner) such as:

- Add city
- Add road
- Delete city
- Delete Road

On the right sidebar he can perform some operations on the graph such as:

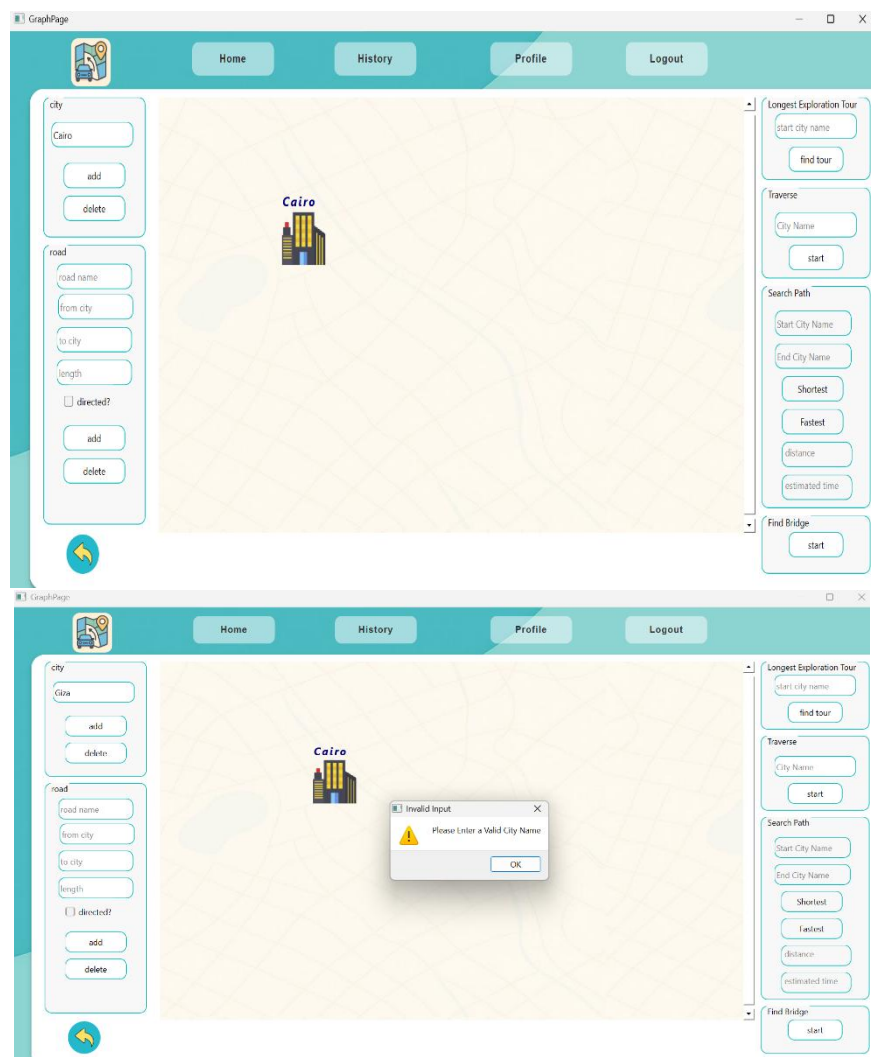
- Find Shortest/Fastest Path between any two existing cities
- Find longest exploration tour starting from a specific node
- Traverse from a specific node (Viewed in the BFS form)
- Find all bridges in the graph

On add city:

Only name of city is required and must not be duplicate.

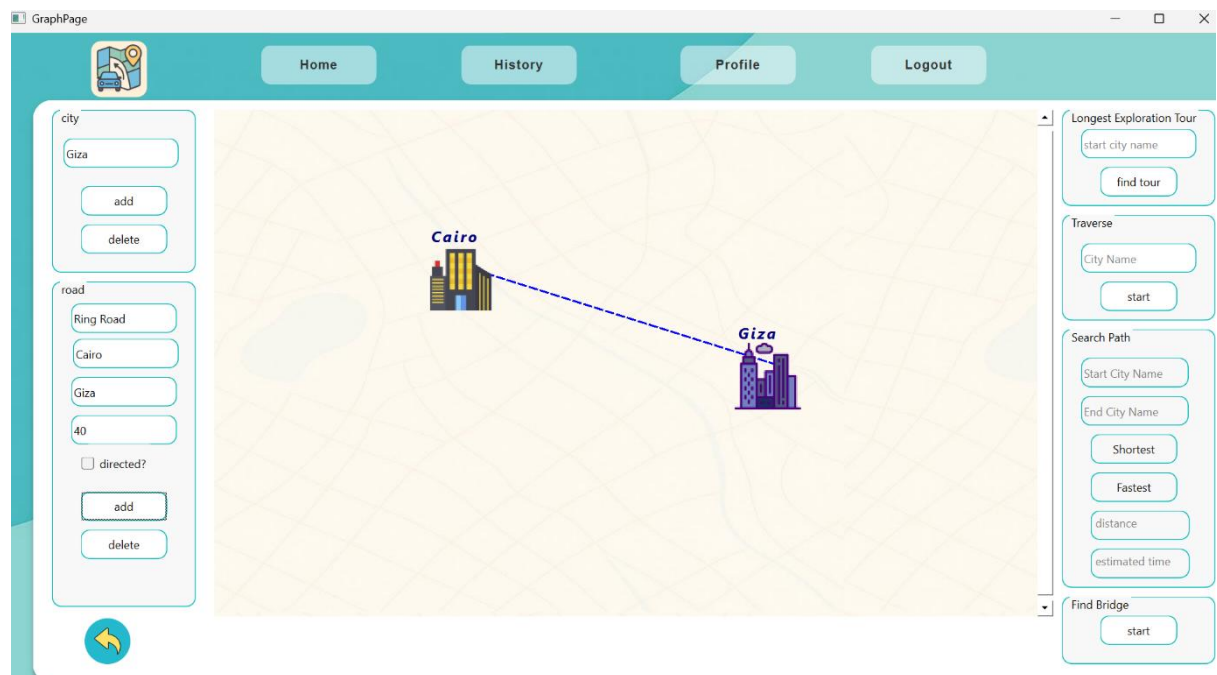
On delete city:

You must enter an existing city name.



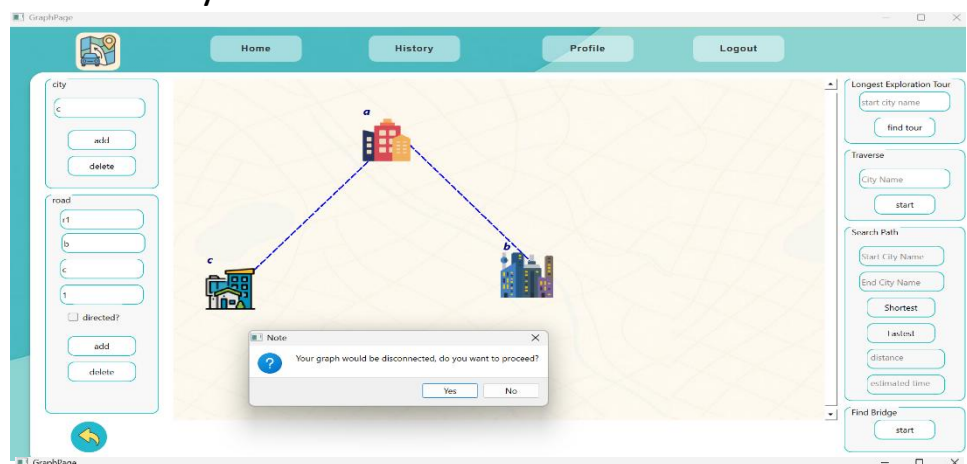
On add road:

You must enter a non-existing road name, two existing citites (source and destination) and the length of this road.

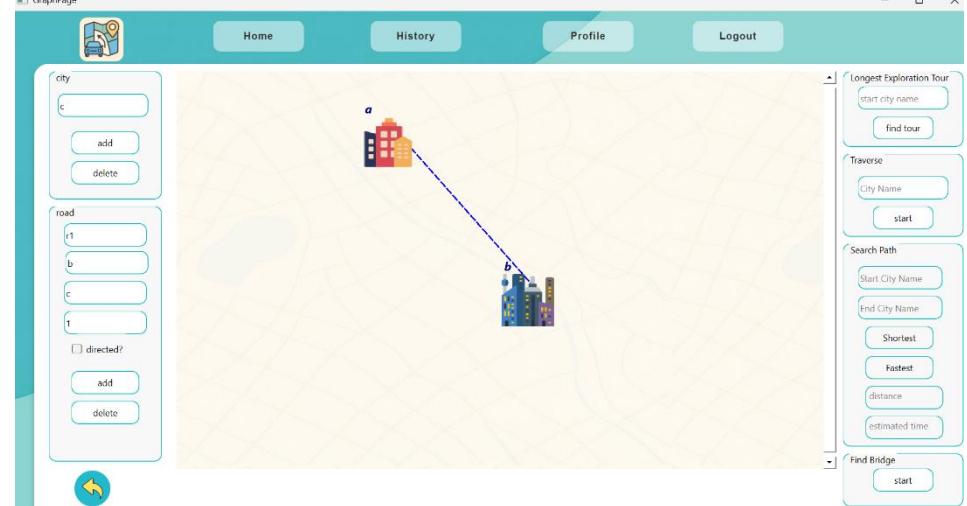


On delete city:

Enter an existing city name, if there is some bridge that will be removed the system will warn you.

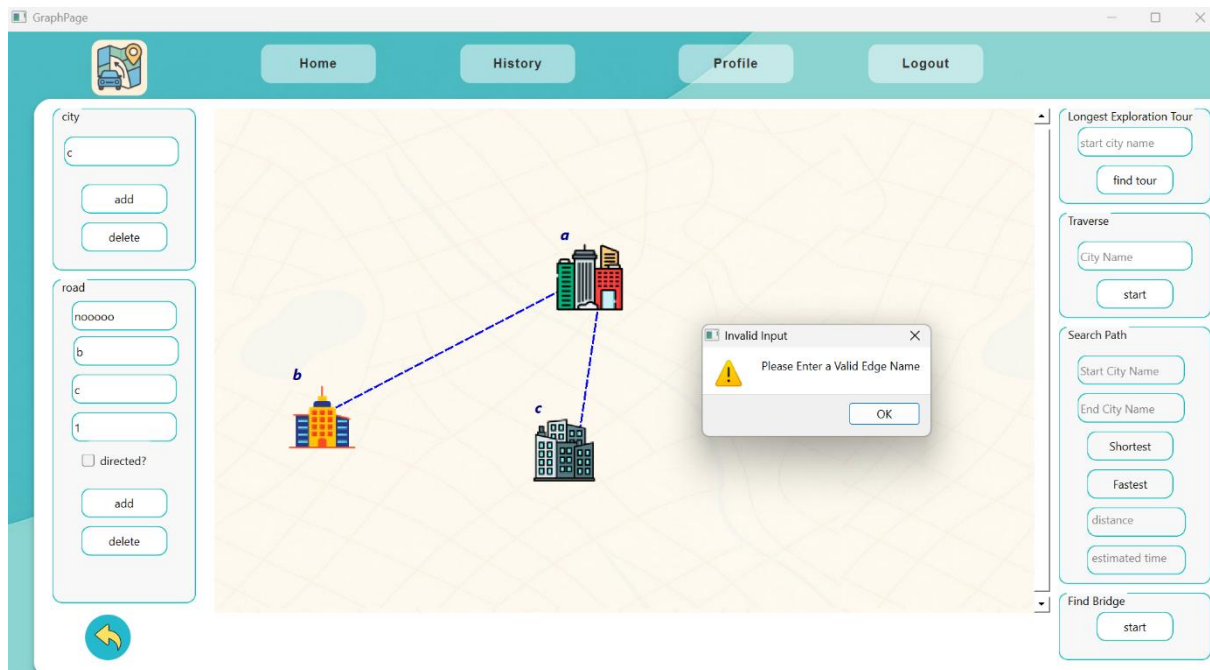


After clicking yes:

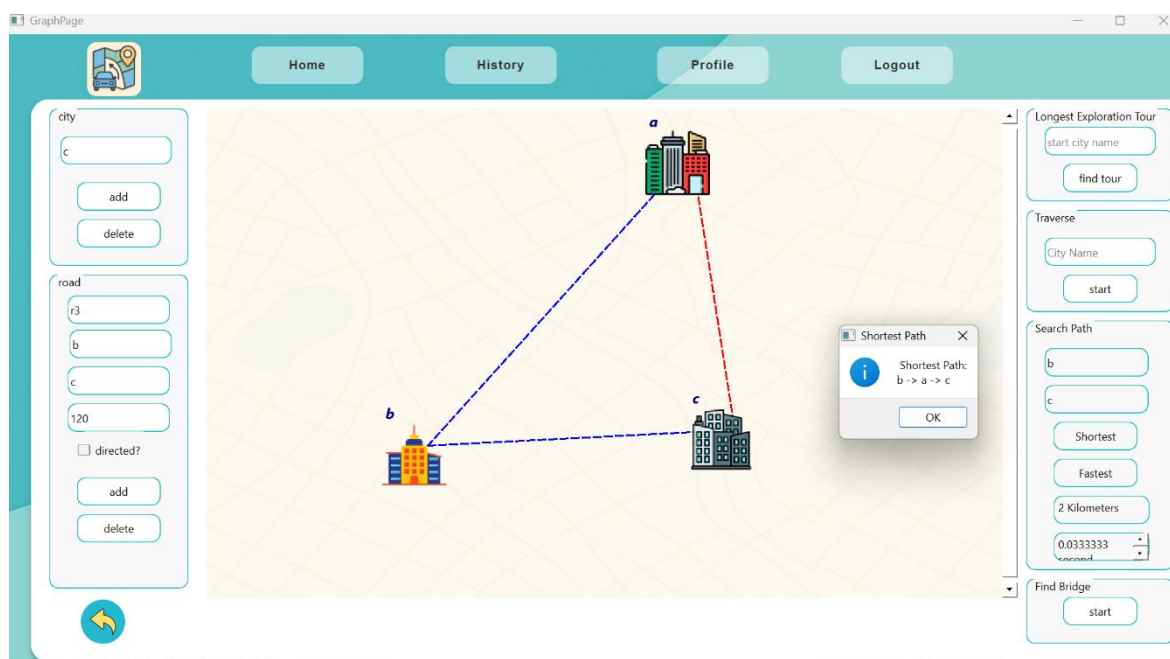


On delete edge:

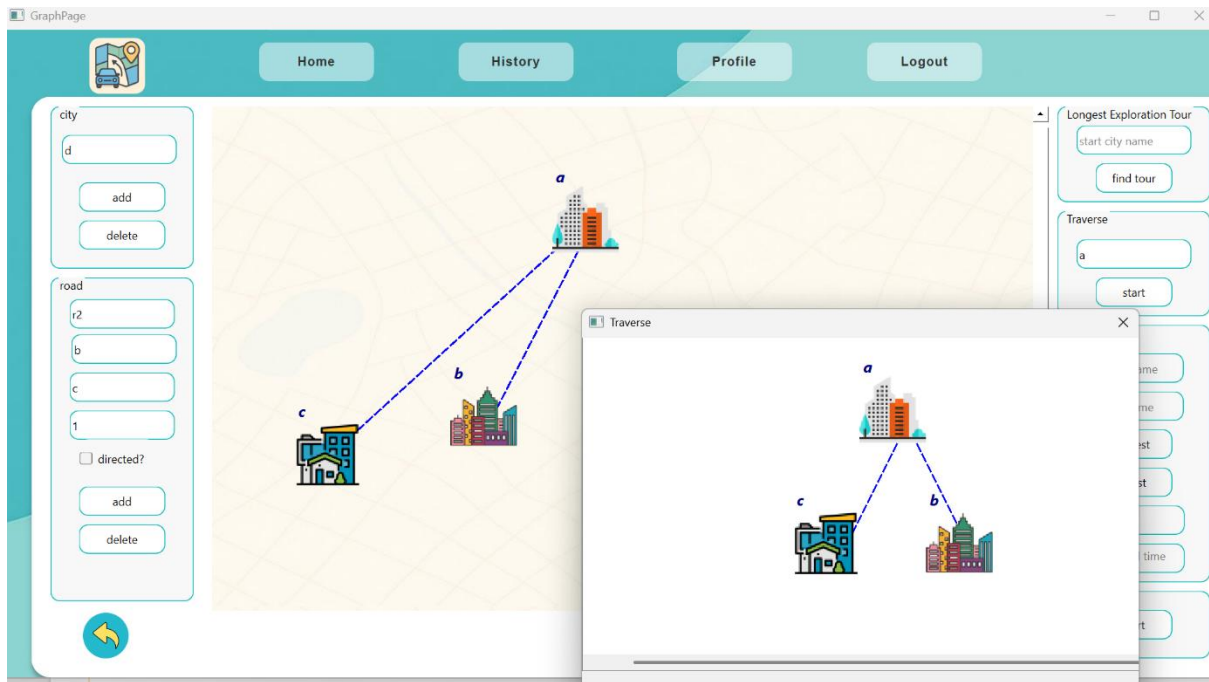
You should enter an existing edge name.



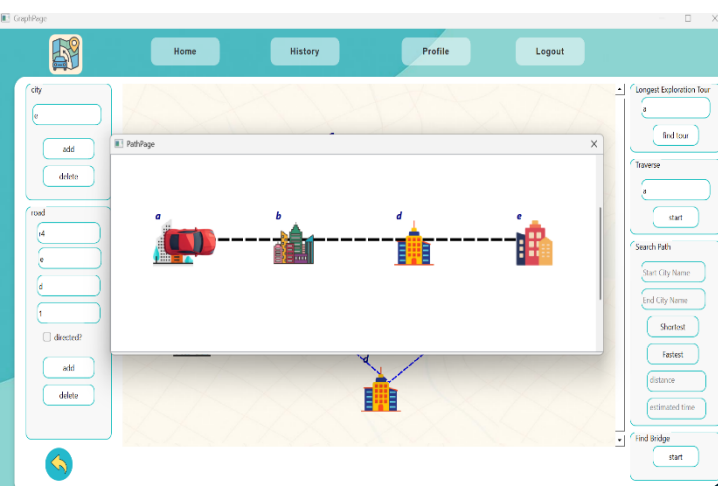
Shortest and Fastest Path (Dijkstra algorithm):



Traverse (BFS Form):

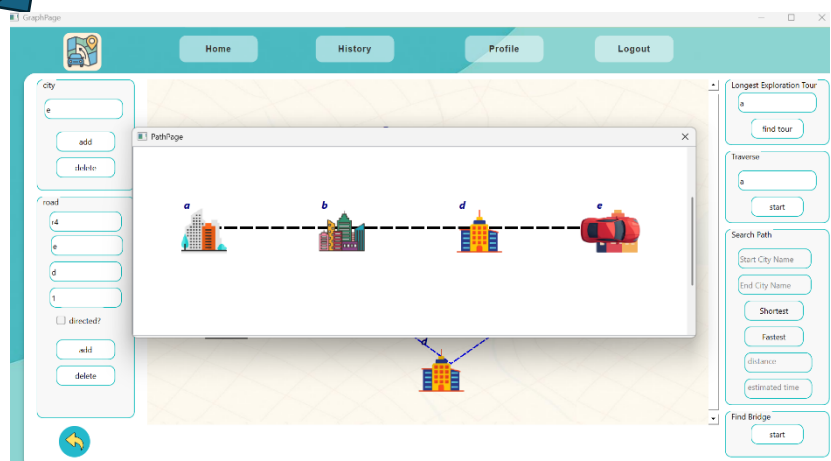


Longest Exploration tour:

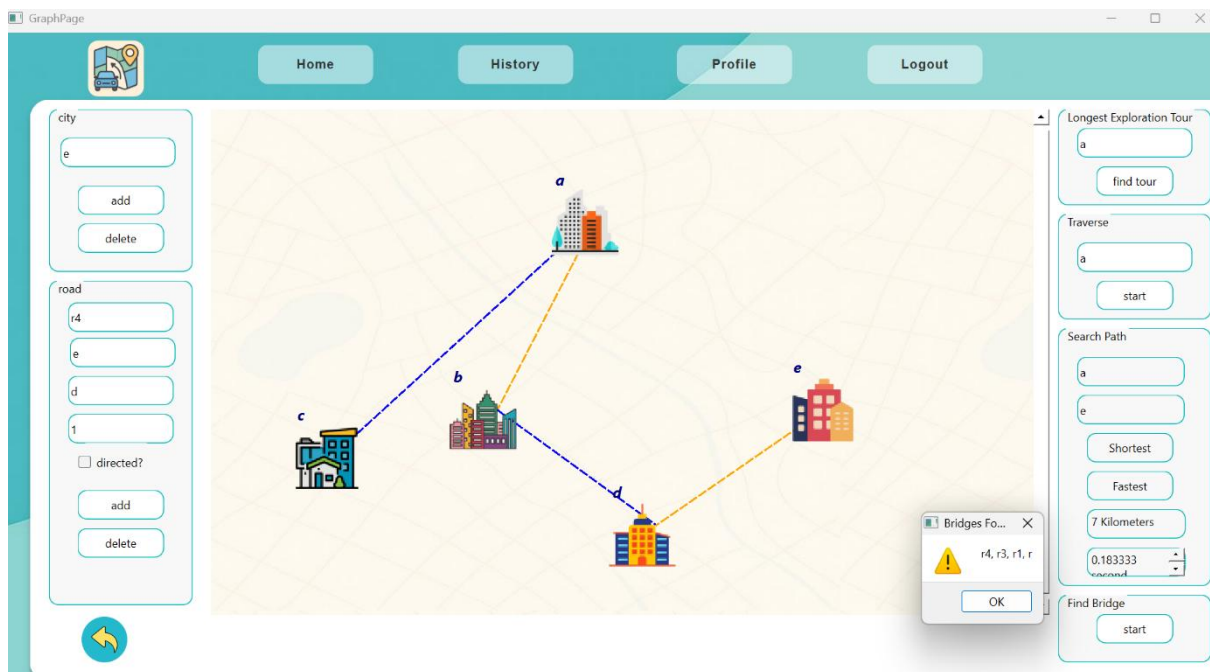


Gets the longest depth using dfs.

You must enter an existing city.

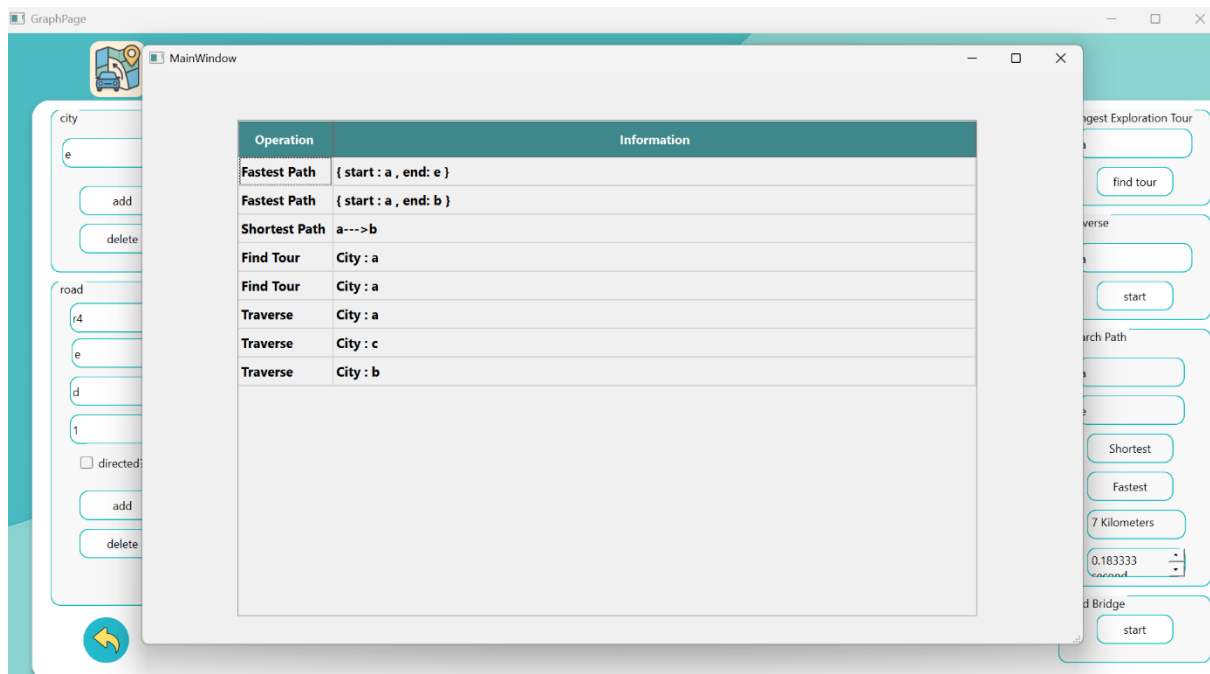


Find bridges:



Returns all the found bridges in the graph.

History:



All past operations can be viewed in the history page.

- **Undo the most recent graph modification operation.**



This function provides undo functionality for the following operations:

1. Adding a city (node): Removes the most recently added city.
2. Adding an edge: Removes the most recently added edge.
3. Deleting a city: Re-adds the most recently deleted city along with its connected edges.
4. Deleting an edge: Re-adds the most recently deleted edge.

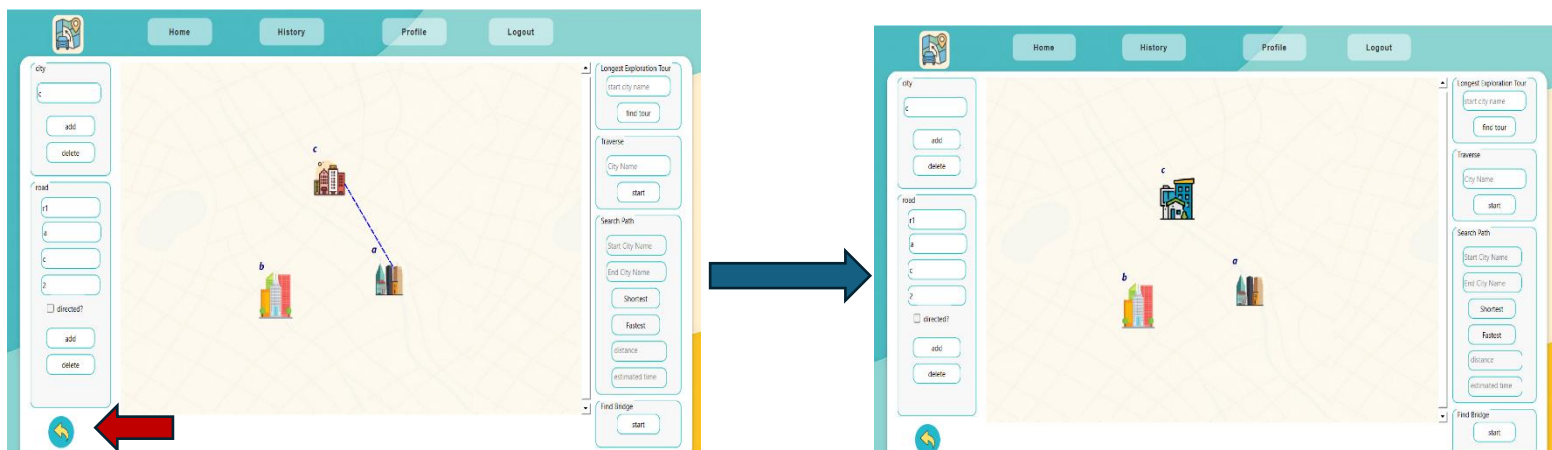
The operation to undo is retrieved from the `'lastOperations'` stack, which stores tuples in the format: (operation type, node name, vector of edges involved).

The function uses a flag “isUndo” to avoid re-pushing operations to

The “lastOperations” stack while performing an undo.

Operation types:

- 1: Add Node
- 2: Add Edge
- 3: Delete Node
- 4: Delete Edge



◆ Features:

- User registration and login system with saved search history
- Graph management: add, update, and delete cities and roads
- BFS traverse
- DFS to find longest exploration tour
- Dijkstra's algorithm for:
 - Shortest path (based on distance)
 - Fastest path (based on traffic factor and search frequency)
- Persistent data storage via files (for users and graphs)
- User full search history
- Qt-based GUI with:
 - Login/Sign-up screens
 - Home page listing available graphs
 - Graph operations: add/delete cities and edges, visualize graph
 - Sidebars showing traversal and path results
 - Path visualization in the GUI using QGraphicsView